

ROTEIRO DE LABORATÓRIO 5

O núcleo da Comanda, do item ao desconto

Aula 7, 29 de setembro de 2026 · herança, composição e polimorfismo · base da entrega parcial da AV1

O que você vai construir

Neste roteiro você monta, do zero e em 10 passos, o núcleo orientado a objetos da Comanda do restaurante: o mesmo exemplo das Aulas 4, 6 e 7. Cada passo acrescenta uma peça, explica por que ela existe e termina com um teste rodando.

Ao final você terá um modelo completo do que a sua equipe entrega hoje. No último passo, uma tabela mostra como trocar a Comanda pelo domínio do projeto de vocês (biblioteca, petshop, barbearia...). Faça na ordem: cada passo depende do anterior.

AO FINAL DESTES ROTEIRO VOCÊ TERÁ

Um pacote domínio/ com as classes Item, Bebida, Prato, três descontos e a Comanda.

Herança só onde cabe (Bebida e Prato são um Item), usando `super()`.

Composição na Comanda (ela tem itens e tem um desconto), sem herdar de `list`.

Polimorfismo: três descontos trocáveis com o mesmo método `aplicar`.

Invariantes protegidas: preço nunca negativo, mesa positiva, comanda fechada não recebe item.

16 testes `pytest` passando e o projeto no GitHub.

Antes de começar

Confira estes itens antes de começar.

- Python 3.10 ou mais recente instalado (`python --version` no terminal).
- VS Code aberto e o terminal integrado funcionando (menu Terminal, New Terminal).
- Git configurado e o repositório da equipe já criado no GitHub (Roteiro 1).
- Lembrar da Aula 6: a Comanda guardava tuplas (nome, preço) em `self._itens` e recusava preço negativo em `adicionar_item`. Hoje o item vira uma classe.

Passo a passo

Em cada passo: leia o "Por que isso importa" antes de digitar, escreva o código (não copie sem ler) e só avance quando o "Confira" estiver certo. No Windows, onde aparecer `python3` use `python`, e onde aparecer `source .venv/bin/activate` use `.venv\Scripts\activate`.

1 Preparar a pasta do projeto

- No terminal, crie a pasta, entre nela e crie um ambiente virtual com o pytest:

```
mkdir comanda && cd comanda
python3 -m venv .venv
source .venv/bin/activate
pip install pytest
mkdir dominio tests
```

POR QUE ISSO IMPORTA

Crie também, pelo VS Code, três arquivos vazios: `dominio/__init__.py`, `tests/__init__.py` e `pytest.ini`. O `__init__.py` transforma a pasta num pacote que pode ser importado. O `pytest.ini` diz ao pytest onde procurar o código.

```
# pytest.ini
[pytest]
pythonpath = .
testpaths = tests
```

```
comanda/
├── pytest.ini
├── dominio/
│   └── __init__.py
└── tests/
    └── __init__.py
```

CONFIRA ANTES DE SEGUIR

O comando `pytest` roda e responde "no tests ran". Isso é o esperado: ainda não escrevemos nenhum teste.

2 A classe Item e a primeira invariante

- Na Aula 6 o item era uma tupla ("água", 6). Agora ele vira uma classe, e a regra "preço nunca negativo" passa a morar nele. Crie dominio/item.py:

```
class Item:
    def __init__(self, nome, preco):
        if not nome or not nome.strip():
            raise ValueError("nome do item não pode ser vazio")
        if preco < 0:
            raise ValueError("preço não pode ser negativo")
        self.nome = nome.strip()
        self.preco = preco

    def descricao(self):
        return f"{self.nome}: R$ {self.preco}"
```

POR QUE ISSO IMPORTA

A validação fica no `__init__`, o ponto de entrada: um Item inválido nem chega a existir. É a mesma ideia da Aula 6 (invariante), agora no dono certo do dado.

```
# tests/test_item.py (comece assim)
import pytest

from dominio.item import Item

def test_item_valido():
    assert Item("água", 6).descricao() == "água: R$ 6"

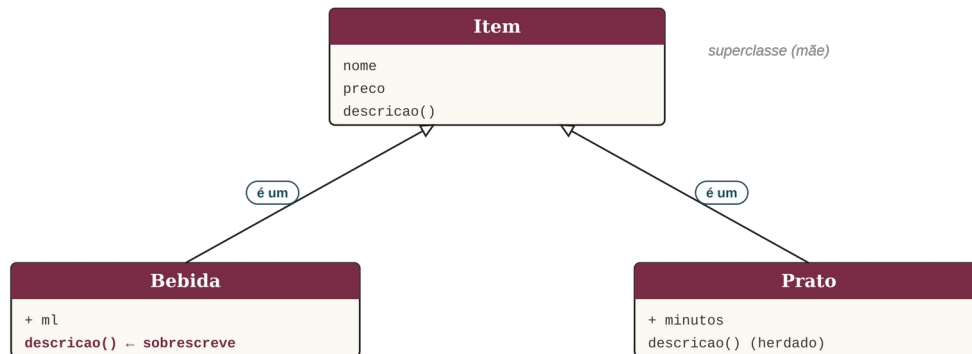
def test_item_com_preco_negativo_levanta_erro():
    with pytest.raises(ValueError):
        Item("erro", -1)
```

CONFIRA ANTES DE SEGUIR

pytest mostra 2 passed. O segundo teste prova que a regra funciona: ele espera o erro.

3 Herança: Bebida e Prato são um Item

- Uma bebida é um item com volume em ml; um prato é um item com tempo de preparo. Os dois passam no teste do "é um". Acrescente no fim de domínio/item.py:

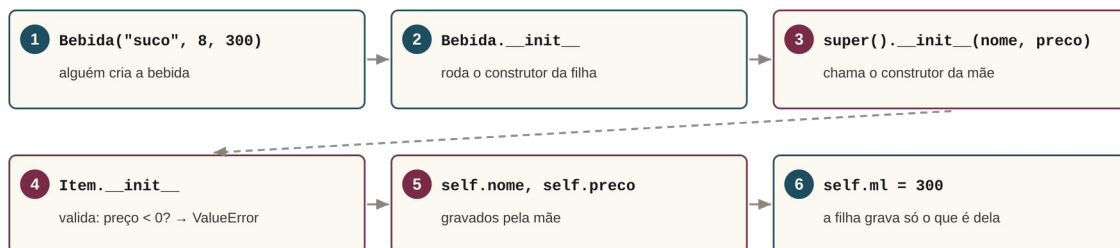


A seta de ponta vazada aponta para a mãe. O que está na mãe, as filhas já têm; o "+" é só o que cada filha acrescenta.

Item é a superclasse (mãe). Bebida e Prato herdam nome, preço e descricao, e acrescentam só o que é deles.

```
class Bebida(Item):
    def __init__(self, nome, preco, ml):
        super().__init__(nome, preco)
        self.ml = ml

class Prato(Item):
    def __init__(self, nome, preco, minutos):
        super().__init__(nome, preco)
        self.minutos = minutos
```



■ código da Bebida (filha) ■ código do Item (mãe), reaproveitado via super()

Sem o passo 3, os passos 4 e 5 nunca acontecem: a bebida nasce sem nome, sem preço e sem validação.

O que `super().__init__` faz: entrega para o Item o trabalho comum, inclusive a validação.

ATENÇÃO

Se você esquecer a linha `super().__init__(nome, preco)`, a Bebida nasce sem nome, sem preço e, pior, sem a validação. Teste isso no próximo passo.

CONFIRA ANTES DE SEGUIR

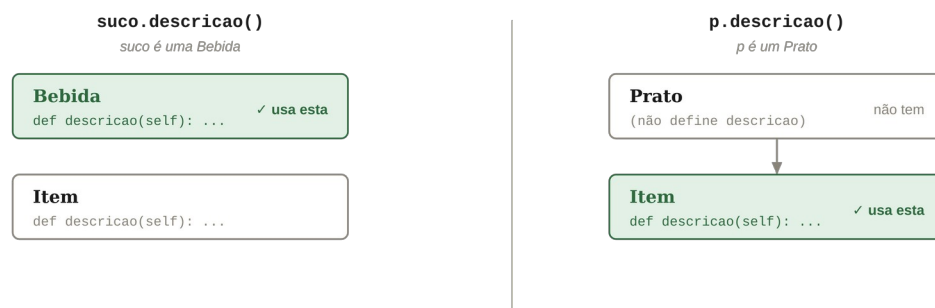
No terminal, `python3 -c "from dominio.item import Bebida; print(Bebida('suco', 8, 300).ml)"` mostra 300.

4 Sobrescrita: cada item se descreve do seu jeito

- A descrição da bebida deve mostrar os ml, e a do prato, o tempo. Cada subclasse redefine `descricao`, reaproveitando a versão da mãe com `super()`. Acrescente dentro de cada classe:

```
# dentro de class Bebida
def descricao(self):
    base = super().descricao()
    return f"{base} ({self.ml} ml)"

# dentro de class Prato
def descricao(self):
    base = super().descricao()
    return f"{base} (~{self.minutos} min)"
```



O Python procura o método primeiro na classe do objeto e, se não achar, sobe para a mãe.

Por isso a `Bebida` mostra os ml e o `Prato` mostra só nome e preço, como no exercício.

O Python procura o método na classe do objeto e, se não achar, sobe para a mãe.

POR QUE ISSO IMPORTA

Complete `tests/test_item.py` com o arquivo abaixo. Repare no teste que prova que a validação da mãe vale para a filha: é ele que garante que o `super()` está lá.

```

import pytest

from dominio.item import Bebida, Item, Prato

def test_item_valido():
    item = Item("água", 6)
    assert item.descricao() == "água: R$ 6"

def test_item_com_preco_negativo_levanta_erro():
    with pytest.raises(ValueError):
        Item("erro", -1)

def test_item_sem_nome_levanta_erro():
    with pytest.raises(ValueError):
        Item(" ", 5)

def test_bebida_herda_validacao_da_mae():
    # super().__init__ garante que a regra de Item vale para Bebida
    with pytest.raises(ValueError):
        Bebida("suco", -8, 300)

def test_bebida_sobrescreve_descricao_reaproveitando_a_mae():
    assert Bebida("suco", 8, 300).descricao() == "suco: R$ 8 (300 ml)"

def test_prato_e_um_item():
    assert isinstance(Prato("feijoadá", 32, 25), Item)

```

CONFIRA ANTES DE SEGUIR

pytest mostra 6 passed.

5 Composição: a Comanda tem itens

- A Comanda não é um item: ela tem itens. Por isso ela guarda uma lista dentro de si, e a única porta de entrada é adicionar_item. Crie dominio/comanda.py (primeira versão):



Teste rápido: leia a frase em voz alta. Se "X é um Y" soar estranho, a relação é "tem um".

Herança é "é um" (seta vazada). Composição é "tem um" (losango).

```
from .item import Item

class Comanda:
    def __init__(self, mesa):
        if mesa <= 0:
            raise ValueError("número da mesa deve ser positivo")
        self.mesa = mesa
        self._itens = []      # composição: tem uma lista

    def adicionar_item(self, item):
        if not isinstance(item, Item):
            raise TypeError("só é possível adicionar um Item")
        self._itens.append(item)

    @property
    def subtotal(self):
        return sum(i.preco for i in self._itens)
```

CONFIRA ANTES DE SEGUIR

Numa sessão python3 no terminal: `from dominio.comanda import Comanda; from dominio.item import Item; c = Comanda(5); c.adicionar_item(Item("água", 6)); c.adicionar_item(Item("prato", 32)); c.adicionar_item(Item("sobremesa", 14)); print(c.subtotal)` mostra 52, o mesmo total da Aula 6.

6 Experimento: por que não herdar de list

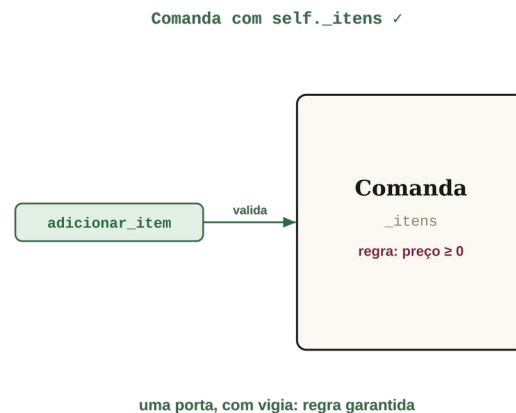
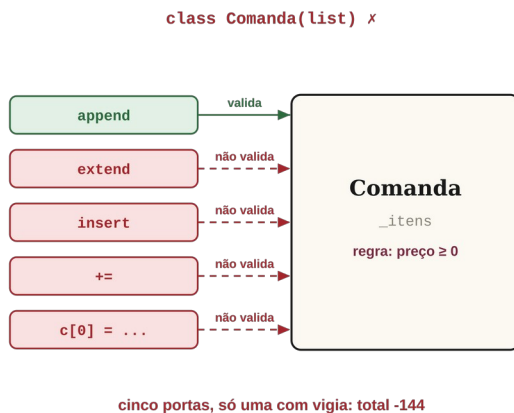
- Parece mais fácil fazer class Comanda(list) e ganhar tudo de graça. Veja o que acontece. Crie um arquivo experimento.py na raiz, rode com python3 experimento.py e depois apague o arquivo:

```
from dominio.item import Item

class ComandaErrada(list):
    def append(self, item):
        if item.preco < 0:
            raise ValueError("preço não pode ser negativo")
        super().append(item)

class Falso:      # objeto qualquer, sem validação
    def __init__(self, preco):
        self.preco = preco

c = ComandaErrada()
c.append(Item("água", 6))
c.extend([Falso(-100)]) # não passa pelo append
c.insert(0, Falso(-50)) # também não
print(sum(i.preco for i in c)) # -144
```



Herdando de list, a Comanda ganha várias portas de entrada; só uma tem a validação.

POR QUE ISSO IMPORTA

Quem herda recebe a interface inteira da mãe, inclusive o que não queria. Compondo, a classe decide quais portas existem. Por isso a nossa Comanda do passo 5 tem só adicionar_item.

CONFIRA ANTES DE SEGUIR

O experimento mostrou -144 e você apagou o experimento.py. A regra "preço nunca negativo" quebrou sem nenhum erro: é exatamente o que evitamos.

7 Polimorfismo: três descontos, um verbo

- O restaurante quer três tipos de desconto. Em vez de uma cadeia de ifs dentro da Comanda, cada desconto vira uma classe com o mesmo método aplicar. Crie dominio/desconto.py:

```
"""Descontos trocáveis.
```

```
Polimorfismo (Aula 7): as três classes respondem ao mesmo verbo, aplicar,
sem herança entre elas (duck typing). A Comanda não precisa saber qual é qual.
"""
```

```
class SemDesconto:
```

```
    def aplicar(self, subtotal):
        return subtotal
```

```
class Percentual:
```

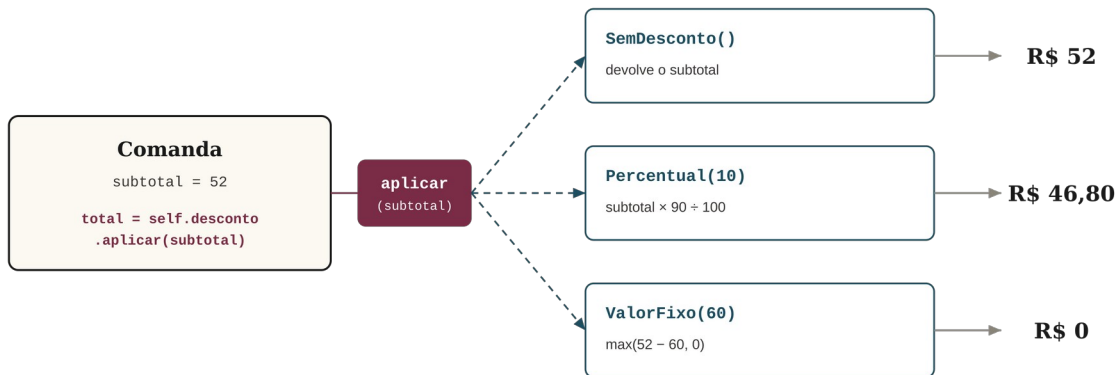
```
    def __init__(self, pct):
        if not 0 <= pct <= 100:
            raise ValueError("percentual deve estar entre 0 e 100")
        self.pct = pct
```

```
    def aplicar(self, subtotal):
        return subtotal * (100 - self.pct) / 100
```

```
class ValorFixo:
```

```
    def __init__(self, valor):
        if valor < 0:
            raise ValueError("valor do desconto não pode ser negativo")
        self.valor = valor
```

```
    def aplicar(self, subtotal):
        # invariante: o total nunca fica negativo
        return max(subtotal - self.valor, 0)
```



Como uma tomada: a Comanda só conhece o formato do plugue (aplicar). Qualquer desconto com esse plugue encaixa.

Como uma tomada: quem chama só conhece o formato do plugue (aplicar).

POR QUE ISSO IMPORTA

Não existe herança entre as três classes. O que as une é responder ao mesmo verbo, com o mesmo contrato: recebe o subtotal, devolve o total. Em Python isso se chama duck typing. Repare que cada uma também protege a própria regra.

CONFIRA ANTES DE SEGUIR

```
python3 -c "from dominio.desconto import Percentual; print(Percentual(10).aplicar(52))" mostra 46.8.
```

8 A Comanda completa: desconto e fechamento

- Agora a Comanda tem também um desconto (composição) e só chama aplicar (polimorfismo). Ganha ainda o fechamento, com duas regras novas. Substitua dominio/comanda.py inteiro por:

```
"""A Comanda de uma mesa.
```

```
Composição (Aula 7): a Comanda TEM itens e TEM um desconto.  
Ela não herda de list: assim, a única porta de entrada é adicionar_item.  
"""
```

```
from .desconto import SemDesconto  
from .item import Item
```

```
class ComandaFechadaError(Exception):  
    """Levantada ao tentar mexer numa comanda já fechada."""
```

```
class Comanda:  
    def __init__(self, mesa, desconto=None):  
        if mesa <= 0:  
            raise ValueError("número da mesa deve ser positivo")  
        self.mesa = mesa  
        self._itens = []          # composição: tem uma lista  
        self.desconto = desconto or SemDesconto() # composição: tem um desconto  
        self._fechada = False
```

```
@property  
def itens(self):  
    # devolve uma cópia: quem está de fora lê, mas não altera  
    return list(self._itens)
```

```
@property  
def fechada(self):  
    return self._fechada
```

```
def adicionar_item(self, item):  
    if self._fechada:  
        raise ComandaFechadaError("comanda fechada não recebe itens")  
    if not isinstance(item, Item):  
        raise TypeError("só é possível adicionar um Item")  
    self._itens.append(item)
```

```
def trocar_desconto(self, desconto):  
    if self._fechada:  
        raise ComandaFechadaError("comanda fechada não muda de desconto")  
    self.desconto = desconto
```

```
def fechar(self):  
    if not self._itens:  
        raise ValueError("não é possível fechar uma comanda vazia")  
    self._fechada = True
```

```
@property
```

```
def subtotal(self):
    return sum(i.preco for i in self._itens)

@property
def total(self):
    return self.desconto.aplicar(self.subtotal)
```

POR QUE ISSO IMPORTA

Três detalhes de projeto: a propriedade `itens` devolve uma cópia (quem está de fora lê, mas não altera a lista); o desconto padrão é `SemDesconto`, então `total` funciona mesmo sem desconto; e `ComandaFechadaError` é uma exceção própria, com nome que explica o problema.

POR QUE ISSO IMPORTA

Por fim, escreva em `dominio/__init__.py` as linhas abaixo. Elas deixam importar tudo direto do pacote:

```
from dominio import Comanda, Item.
```

```
"""Núcleo do domínio da Comanda. Não conhece banco, API nem tela."""
from .item import Item, Bebida, Prato
from .desconto import SemDesconto, Percentual, ValorFixo
from .comanda import Comanda, ComandaFechadaError
```

CONFIRA ANTES DE SEGUIR

```
python3 -c "from dominio import Comanda, Item, ValorFixo; c = Comanda(5, ValorFixo(60));
c.adicionar_item(Item('água', 6)); print(c.total)"
```

 mostra 0: o desconto não deixa o total ficar negativo.

9 Testar a Comanda com todos os descontos

- Crie `tests/test_comanda.py`. Repare no `pytest.mark.parametrize`: o mesmo teste roda uma vez para cada desconto, provando o polimorfismo.

```

import pytest

from dominio import (Comanda, ComandaFechadaError, Item, Percentual,
                     SemDesconto, ValorFixo)

ITENS_AULA_6 = [("água", 6), ("prato", 32), ("sobremesa", 14)]

def comanda_com_itens(desconto=None):
    c = Comanda(mesa=5, desconto=desconto)
    for nome, preco in ITENS_AULA_6:
        c.adicionar_item(Item(nome, preco))
    return c

def test_total_sem_desconto_e_52():
    assert comanda_com_itens().total == 52

@pytest.mark.parametrize("desconto, esperado", [
    (SemDesconto(), 52),
    (Percentual(10), 46.8),
    (ValorFixo(60), 0),
])
def test_total_com_cada_desconto(desconto, esperado):
    assert comanda_com_itens(desconto).total == pytest.approx(esperado)

def test_mesa_invalida_levanta_erro():
    with pytest.raises(ValueError):
        Comanda(mesa=0)

def test_comanda_fechada_nao_recebe_item():
    c = comanda_com_itens()
    c.fechar()
    with pytest.raises(ComandaFechadaError):
        c.adicionar_item(Item("café", 5))

def test_nao_fecha_comanda_vazia():
    with pytest.raises(ValueError):
        Comanda(mesa=1).fechar()

def test_itens_devolve_copia():
    c = comanda_com_itens()
    c.itens.append(Item("intruso", 0)) # mexe só na cópia
    assert len(c.itens) == 3

```

```
def test_so_aceita_item():
    with pytest.raises(TypeError):
        Comanda(mesa=1).adicionar_item(("tupla", 5))

def test_percentual_fora_da_faixa():
    with pytest.raises(ValueError):
        Percentual(150)
```

POR QUE ISSO IMPORTA

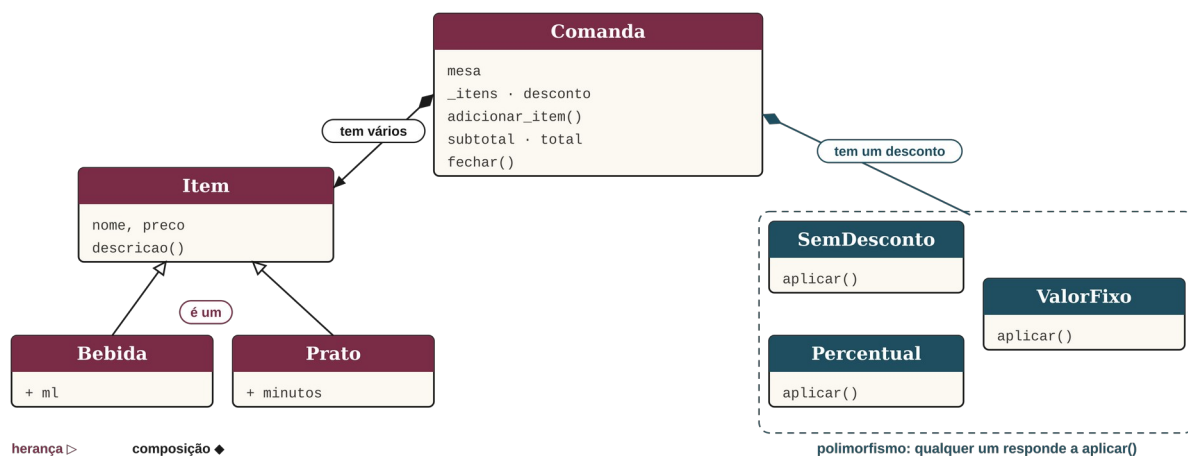
pytest.approx compara números com casas decimais com uma pequena tolerância. Lembre da Aula 3: $52 \times 90 / 100$ em ponto flutuante pode não dar exatamente 46.8.

CONFIRA ANTES DE SEGUIR

pytest -v mostra 16 passed: 6 testes de item e 10 de comanda (o parametrize conta como 3).

10 Conferir, desenhar e entregar

- Compare o seu código com o mapa abaixo. Todas as setas precisam existir no código.
- Crie um README.md curto com o nome do projeto, as classes e como rodar os testes (pip install pytest e pytest).
- Crie um .gitignore com .venv/, __pycache__/ e .pytest_cache/ e envie tudo:



O núcleo completo: quem é um (herança), quem tem um (composição) e quem responde ao mesmo verbo (polimorfismo).

```
git status
git add .
git commit -m "Adiciona núcleo OO com herança, composição e descontos"
git push
```

CONFIRA ANTES DE SEGUIR

No GitHub aparecem as pastas domínio/ e tests/, o README e o último commit. A pasta .venv/ NÃO aparece.

11 Agora no projeto da sua equipe

- Repita os passos 2 a 9 com o domínio de vocês. Use a tabela como guia de tradução:

Na Comanda	Biblioteca	Petshop	Barbearia
Item	Exemplar	Serviço	Serviço
Bebida / Prato	Livro / Revista	Banho / Tosa	Corte / Barba
Comanda	Empréstimo	Agendamento	Atendimento
aplicar (desconto)	calcular (multa)	calcular (taxa extra)	aplicar (desconto)
preço >= 0	prazo > 0	horário no futuro	duração > 0

ATENÇÃO

Não force herança. Se a frase "X é um Y" soar estranha no domínio de vocês, use composição. Na entrega, justifique cada herança com um comentário de uma linha.

CONFIRA ANTES DE SEGUIR

Checklist da entrega parcial: 3 a 5 classes; pelo menos duas invariantes, cada uma com teste que espera o erro; uma composição; herança só se passar no "é um"; um polimorfismo; pytest verde; README; último commit no GitHub até o fim da aula.

Se sobrar tempo

Crie um quarto desconto, HappyHour, que dá 20% só em bebidas. Dica: ele precisa receber a lista de itens, e não só o subtotal. O que isso muda no contrato do método aplicar? Anote a resposta: é o assunto da Aula 8, quando o contrato ganha forma explícita.

Todo o código deste roteiro está no projeto exemplo comanda-exemplo-ate-aula-07, disponível com o professor, para conferência depois que você tentar sozinho.